

JETHVA KARTIK

MODULE – 17

Rest Framework

1. Theory:

1) What is an API (Application Programming Interface)?

Ans. An API (Application Programming Interface) is a set of rules and tools that allows different software applications to communicate with each other. It defines how requests and responses should be structured so one program can access features or data from another — like a waiter taking your order to the kitchen and bringing back your food.

2. Types of APIs: REST, SOAP.

Ans. Representational State Transfer:

- **Style:** Architectural style, not a protocol
- **Data Format:** Typically JSON or XML
- **Transport:** Uses standard HTTP methods (GET, POST, PUT, DELETE)
- **Flexibility:** Lightweight, easy to use, and scalable
- **Stateless:** Each request is independent
- **Best for:** Web and mobile applications

Simple Object Access Protocol:

- **Style:** Protocol with strict standards
- **Data Format:** Only XML
- **Transport:** Can use HTTP, SMTP, TCP, etc.
- **Security:** Built-in standards like WS-Security
- **Strict:** Requires a WSDL (Web Services Description Language)
- **Best for:** Enterprise-level applications needing high security and ACID compliance

3. Why are APIs important in web development?

Ans. APIs are crucial in web development because they enable seamless interaction between different software systems.

Integration Made Easy

- **APIs allow websites to connect with external services like payment gateways (e.g., Stripe), maps (e.g., Google Maps), or social logins (e.g., Facebook, Google).**
- **This reduces development time and complexity.**

Faster Development

- **Developers can reuse existing APIs instead of building features from scratch.**
- **Speeds up time-to-market and encourages modular design.**

Data Exchange

- **APIs enable real-time data sharing between frontend and backend, or between different apps.**
- **Essential for dynamic web apps like dashboards, e-commerce, or social platforms.**

Scalability and Flexibility

- **APIs decouple frontend from backend, allowing teams to scale or update parts independently.**
- **Supports micro services architecture and mobile app integration.**

Security and Control

- **APIs expose only necessary data and functions, keeping internal systems protected.**

- Authentication and rate limiting help manage access.

4. Understanding project requirements.

Ans. Project requirements define what the project must achieve, including:

- **Functional needs:** What the system should do (e.g., user login, data storage)
- **Non-functional needs:** How it should perform (e.g., speed, security, scalability)
- **Business goals:** Why the project exists (e.g., increase sales, improve UX)

Why It's Important

- Prevents scope creep and miscommunication
- Aligns stakeholders and developers
- Guides design, development, and testing
- Saves time and cost by avoiding rework

How to Gather Them

- Stakeholder interviews and questionnaires
- User stories and use cases
- Competitive analysis and benchmarking
- Technical feasibility studies

What to Look For

- Clear objectives and success criteria
- Target audience and user personas
- Platform and technology constraints
- Timeline, budget, and resource limits

5. What is Serialization?

Ans. Serialization is the process of converting data or an object into a format that can be easily stored or transmitted — such as JSON, XML, or binary — and later reconstructed (deserialized) back into its original form.

- **Data transfer:** Send objects over a network (e.g., API responses)
- **Storage:** Save objects to files or databases
- **Caching:** Store pre-processed data for faster access

6. Converting Django QuerySets to JSON.

Ans.

- **JSON is lightweight, readable, and widely supported across platforms.**
- **It's the standard format for data exchange in REST APIs.**
- **Frontend frameworks like React or Vue expect JSON responses.**

7. Using serializers in Django REST Framework (DRF).

Ans. A serializer is like a translator between your Django models and the outside world (usually JSON). It:

- **Converts model data into JSON (serialization)**
- **Validates and converts incoming JSON into model data (deserialization)**

DRF Serializer : Automatic field mapping from models

- Validation of incoming data
- Customizable for complex or nested data
- Works seamlessly with DRF views and routers

8. HTTP request methods (GET, POST, PUT, DELETE).

Ans. GET

- Purpose: Retrieve data from the server
- Example: Fetching a list of notes or user profiles
- Safe & Idempotent: Doesn't change server data

POST

- Purpose: Send new data to the server
- Example: Submitting a form or creating a new note
- Not Idempotent: Each request may create a new resource

PUT

- Purpose: Update existing data completely
- Example: Replacing a note's title and content
- Idempotent: Repeating the request has the same effect

DELETE

- Purpose: Remove data from the server
- Example: Deleting a user or a note
- Idempotent: Repeating the request still results in deletion

9. Sending and receiving responses in DRF.

Ans. Sending Responses

DRF uses the Response class to send structured data back to the client.

- **Format:** Typically JSON, but can also be XML or others depending on configuration
- **Contents:** Can include serialized data, status codes, and messages
- **Status Codes:** Indicate success or failure (e.g., 200 OK, 201 Created, 400 Bad Request)

This makes it easy to return clean, consistent API responses.

Receiving Requests

DRF views handle incoming HTTP requests (GET, POST, PUT, DELETE) and process the data.

- **Data Access:** Use `request.data` to access incoming JSON or form data
- **Validation:** Serializers check if the data is correct and complete
- **Actions:** Based on the method, DRF can create, update, retrieve, or delete resources

Typical Flow

- **Client sends a request (e.g., POST with new note data)**
- **DRF view receives it, passes it to a serializer**
- **Serializer validates and saves the data**
- **DRF sends back a response with the result and status code**

10. Understanding views in DRF: Function-based views vs Class-based views.

Ans. Function-Based Views (FBVs)

- **Structure:** Simple Python functions
- **Flexibility:** Easy to understand and quick to write
- **Control:** You manually handle request methods (GET, POST, etc.)
- **Best for:** Small, straightforward endpoints or when you need fine-grained control

Example use case: A custom login endpoint or a simple data fetch

Class-Based Views (CBVs)

- **Structure:** Python classes that inherit from DRF base views
- **Modularity:** Organize logic into methods like `get()`, `post()`, `put()`, etc.
- **Powerful:** Built-in generic views (like `ListAPIView`, `CreateAPIView`) reduce boilerplate
- **Best for:** Scalable APIs with reusable patterns and complex logic

Example use case: CRUD operations for models like Notes, Users, or Products

When to Use What

- Use FBVs for quick prototypes or custom behavior
- Use CBVs for production-grade APIs with consistent patterns

11. Defining URLs and linking them to views.

Ans. What Are URLs in Django?

URLs are the entry points to your application. Each URL pattern maps a specific path (like `/notes/` or `/users/`) to a view that handles the request.

Linking URLs to Views

You define URL patterns in a file called `urls.py`. Each pattern connects:

- A path (what the user types or the frontend calls)
- A view (the logic that runs when that path is accessed)

In DRF, views can be:

- Function-based views: Linked using `path()` or `re_path()`
- Class-based views: Linked using `as_view()` method
- ViewSets: Linked using DRF's `DefaultRouter` for automatic routing

Why It Matters

- It organizes your app's structure
- Makes your API endpoints discoverable and testable
- Allows frontend and external systems to interact with your backend

Example Use Cases

- `/api/notes/` → Returns a list of notes
- `/api/notes/5/` → Returns or updates a specific note
- `/api/users/login/` → Handles user login

12. Adding pagination to APIs to handle large data sets

Ans. Why It's Important

- **Performance:** Reduces server load and response time
- **Efficiency:** Limits bandwidth usage, especially for mobile clients
- **User Experience:** Enables infinite scroll, page navigation, or “load more” features

How DRF Handles Pagination

Django REST Framework offers built-in pagination classes:

- **PageNumberPagination:** Clients request specific pages (e.g., `?page=2`)
- **LimitOffsetPagination:** Clients specify how many items and where to start (e.g., `?limit=10&offset=20`)
- **CursorPagination:** More secure and efficient for large, changing datasets

You can set a default pagination style in `settings.py` or customize it per view.

What Clients Receive

Paginated responses typically include:

- The current page of results
- Metadata like total count, next/previous page links
- Navigation parameters for fetching more data

13. REST principles: statelessness, resource-based URLs, and using HTTP methods for CRUD operations.

Ans. Statelessness

- Every request from the client to the server must contain all the information needed to understand and process it.
- The server does not store any session or context between requests.
- This makes REST APIs scalable and easier to cache, distribute, and debug.

Example: Each API call includes authentication tokens and all necessary data — no reliance on previous calls.

Resource-Based URLs

- REST treats everything as a resource (e.g., users, notes, products).
- URLs should clearly represent these resources using nouns, not actions.

Examples:

- `/api/notes/` → list or create notes
- `/api/notes/5/` → retrieve, update, or delete note with ID 5

This approach makes APIs intuitive and consistent.

Using HTTP Methods for CRUD

REST maps standard HTTP methods to CRUD operations:

HTTP Method	CRUD Operation	Purpose
GET	Read	Fetch data
POST	Create	Add new data
PUT	Update	Replace existing data
PATCH	Update	Modify part of the data
DELETE	Delete	Remove data

This alignment keeps APIs predictable and aligned with web standards.

14. What is CRUD, and why is it fundamental to backend development?

Ans. CRUD stands for Create, Read, Update, Delete — the four basic operations that most applications perform on data. It's fundamental to backend development because it defines how your server interacts with the database and manages resources.

What Each Operation Does

- **Create:** Add new data (e.g., a new user or note)
- **Read:** Retrieve existing data (e.g., view a list of notes)
- **Update:** Modify existing data (e.g., edit a note's title)
- **Delete:** Remove data (e.g., delete a user account)

Why CRUD Is Essential

- **Core of data management:** Every backend system revolves around storing and manipulating data.

- **Maps directly to HTTP methods: RESTful APIs use POST (Create), GET (Read), PUT/PATCH (Update), DELETE (Delete).**
- **Universal pattern: Whether you're building a blog, e-commerce site, or notes app, CRUD operations are the foundation.**
- **Scalability: CRUD logic can be extended to handle complex workflows, permissions, and validations.**

15. Difference between authentication and authorization.

Ans. Authentication

- **What it is: Verifying who the user is.**
- **Purpose: Confirms identity using credentials like username/password, tokens, or biometrics.**
- **Example: Logging into a notes app with your email and password.**

Authorization

- **What it is: Determining what the authenticated user is allowed to do.**
- **Purpose: Controls access to resources or actions based on roles or permissions.**
- **Example: Allowing only admins to delete notes or access user data.**

Key Differences

Feature	Authentication	Authorization
Focus	Identity	Permissions
Happens First	✅ Yes	❌ Only after authentication
Determines	Who you are	What you can do
Example	Login with credentials	Access admin dashboard

16. Implementing authentication using Django REST Framework's token-based system.

Ans. What Is Token-Based Authentication?

- Each user is issued a unique token after logging in.
- This token is sent with every API request (usually in the header).
- The server checks the token to verify the user's identity.

Why Use It?

- **Stateless:** No need to store session data on the server
- **Mobile-friendly:** Works well with mobile and single-page apps
- **Secure:** Tokens can be revoked or rotated

How It Works in DRF

- User logs in via a special endpoint (e.g., /api-token-auth/)
- DRF returns a token if credentials are valid
- Client stores the token (e.g., in localStorage or memory)
- Client sends the token with each request in the Authorization header
- DRF authenticates the request using the token

Token Lifecycle

- **Tokens are typically long-lived unless manually revoked**
- **You can customize token expiration or use alternatives like JWT for more control**

